BrainAI Logo

ProSuite Industrial

WASM & CLOUD DEPLOYMENT SPEC

PRODUCT DOCUMENTATION

BrainAI (Inhaber: Sascha Alexander Köhne)

BrainAI Inhaber: Sascha Alexander Köhne	PROSUITE INDUSTRIAL	🟢 PUBLIC CLASSIFICATION
Core Architecture & Development	**Server Integration & Hardening Guide**	Access Level: Public / Beta

---

Revisionshistorie / Revision History

Version	Datum	Autor	Status	Beschreibung
0.9	15.01.2026	S. Köhne	Draft	Initialer Entwurf (BioEDC Architektur)
1.0	20.02.2026	S. Köhne	Release	Rebranding auf ProEDC & Industrial Hardening

🇩🇪 Deutsch (de_de) | 🇬🇧 English (int_eng)

🇩🇪 ProEDC Industrial Suite | Server Integration Guide

Version: 1.0
Fokus: IIS, Apache, Nginx & Docker Hardening
Zielgruppe: Systemadministratoren / DevOps / IT-Ops

1. Allgemeine Systemanforderungen

Die ProEDC Industrial Suite ist eine **Client-Side-Only** Anwendung. Die gesamte kryptographische Logik wird im Browser des Anwenders via WebAssembly (WASM) ausgeführt. Es wird keine serverseitige Skriptsprache (wie PHP, Python, ASP.NET) benötigt.

Damit die Anwendung jedoch performant (LCP < 1s) und sicher ausgeführt werden kann, muss der Webserver drei spezifische Anforderungen erfüllen:

A. MIME-Type Konfiguration

WebAssembly-Dateien (`.wasm`) müssen mit dem korrekten MIME-Type `application/wasm` ausgeliefert werden.

- *Fehlerbild:* Der Browser verweigert das Laden der App oder zeigt Konsolenfehler ("Incorrect response MIME type").

B. Security Headers (COOP / COEP)

Für kryptographische Hochleistungsoperationen nutzt ProEDC moderne Speicherstrukturen (`SharedArrayBuffer`). Moderne Browser (Chrome, Edge,

Firefox) erfordern dafür eine strikte Isolation der Webseite.

- **Cross-Origin-Opener-Policy:** `same-origin`
- **Cross-Origin-Embedder-Policy:** `require-corp`

C. Kompression (Brotli / Gzip)

Die Binärdatei `proedc.wasm` ist optimiert, aber unkomprimiert. Der Server sollte **statische Kompression** aktivieren, um die Datei für die Übertragung drastisch zu verkleinern.

---

2. Microsoft IIS (Internet Information Services)

In Windows-Server-Umgebungen erfolgt die Konfiguration am einfachsten über eine `web.config` Datei. Diese weist den IIS an, die `.wasm` Datei korrekt zu behandeln, ohne dass globale Server-Einstellungen geändert werden müssen.

Installationsanweisung

1. Kopieren Sie den Inhalt des `wasm`-Ordners in das Webroot (z.B. `C:\inetpub\wwwroot\proedc`).
2. Erstellen Sie im selben Verzeichnis eine Datei namens `web.config`.
3. Fügen Sie den folgenden XML-Code ein.

Konfiguration: `web.config`

```xml
<?xml version="1.0" encoding="UTF-8"?>
<configuration>
  <system.webServer>

    <staticContent>
      <remove fileExtension=".wasm" />
      <mimeMap fileExtension=".wasm" mimeType="application/wasm" />
      <remove fileExtension=".json" />
      <mimeMap fileExtension=".json" mimeType="application/json" />
    </staticContent>
```

```
    <httpProtocol>
      <customHeaders>
        <add name="Cross-Origin-Opener-Policy" value="same-origin" />
        <add name="Cross-Origin-Embedder-Policy" value="require-corp"

        <add name="X-Content-Type-Options" value="nosniff" />
        <add name="X-Frame-Options" value="SAMEORIGIN" />
        <add name="Strict-Transport-Security" value="max-age=31536000
      </customHeaders>
    </httpProtocol>

    <urlCompression doStaticCompression="true" doDynamicCompression="

    <caching>
      <profiles>
        <add extension=".wasm" policy="CacheUntilChange" kernelCacheP
        <add extension=".js" policy="CacheUntilChange" kernelCachePol
      </profiles>
    </caching>

  </system.webServer>
</configuration>
```

Verifikation der IIS-Installation

Um sicherzustellen, dass der IIS korrekt konfiguriert ist:

1. Öffnen Sie die URL im Browser (Edge/Chrome).
2. Drücken Sie `F12` für die Entwickler-Tools -> Reiter **Netzwerk**.
3. Laden Sie die Seite neu.
4. Klicken Sie auf die Datei `proedc.wasm` in der Liste.
5. Prüfen Sie unter **Antwortheader (Response Headers)**:

- `Content-Type`: Muss `application/wasm` lauten.
- `Cross-Origin-Embedder-Policy`: Muss `require-corp` lauten.

---

3. Apache HTTP Server (.htaccess)

Der Apache Webserver ist in vielen Linux-basierten Intranets und Shared-Hosting-Umgebungen der Standard. Die Konfiguration erfolgt über eine `.htaccess` Datei im Webroot.

Installationsanweisung

1. Kopieren Sie den Inhalt des `wasm`-Ordners in das Webroot (z.B. `/var/www/html/proedc`).
2. Erstellen Sie im selben Verzeichnis eine Datei namens `.htaccess` (Punkt am Anfang beachten!).
3. Fügen Sie den bereitgestellten Konfigurations-Code ein.

Wichtige Voraussetzungen: Stellen Sie sicher, dass in der Hauptkonfiguration (`httpd.conf`) folgende Module aktiviert sind (meistens Standard):

- `mod_headers` (für Security Header)
- `mod_mime` (für WASM Typerkennung)
- `mod_deflate` (für Kompression)
- `mod_expires` (für Caching)

Funktionsweise der Konfiguration

- **WASM Isolation:** Die Header `Cross-Origin-Opener-Policy` und `Cross-Origin-Embedder-Policy` werden gesetzt. Dies erlaubt dem Browser, `SharedArrayBuffer` zu nutzen, was die Krypto-Performance um bis zu 30% steigert.
- **Compression:** Die Datei `proedc.wasm` wird vor dem Senden komprimiert. Dies reduziert die Übertragungszeit im Schnitt um 60-70%.
- **Lockdown:** Der Zugriff auf interne Dateien (wie `.git` Ordner oder Makefiles), die versehentlich hochgeladen wurden, wird blockiert.

---

4. Nginx (High Performance & Linux)

Nginx ist die empfohlene Umgebung für Produktions-Deployments mit hoher Last. Diese Konfiguration ist aggressiv auf die schnelle Auslieferung von WebAssembly-Streams optimiert.

Installationsanweisung

1. Erstellen Sie eine Datei `nginx.conf` in Ihrem Konfigurationsverzeichnis (oder nutzen Sie dieses Snippet in Ihrer `sites-available`).
2. Stellen Sie sicher, dass der Pfad `root` auf Ihren `wasm`-Ordner zeigt.

Konfiguration: `nginx.conf` (Optimiert)

```
# Worker Processes auf "auto" nutzt alle CPU-Kerne für I/O
worker_processes auto;

events {
    worker_connections 1024;
    multi_accept on;
}

http {
    include /etc/nginx/mime.types;
    default_type application/octet-stream;

    # 1. WASM MIME-TYPE SUPPORT (Kritisch)
    types {
        application/wasm wasm;
    }

    # 2. PERFORMANCE TUNING
    sendfile on;
    tcp_nopush on;
    tcp_nodelay on;
    keepalive_timeout 65;
    server_tokens off; # Versteckt Version (Security by Obscurity)

    # 3. GZIP COMPRESSION
    # Reduziert die Ladezeit der .wasm Datei um ca. 60-70%
    gzip on;
    gzip_vary on;
    gzip_proxied any;
    gzip_comp_level 6; # Balance zwischen CPU-Last und Kompression
    gzip_types text/plain text/css application/json application/javas

    server {
        listen 80;
```

```
        server_name localhost;
        root /usr/share/nginx/html; # Pfad zum ProEDC 'wasm' Ordner
        index index.html;

        # 4. SECURITY HEADERS (Industrial Hardening)
        # Erforderlich für SharedArrayBuffer (WASM Threading Support)
        add_header Cross-Origin-Opener-Policy "same-origin" always;
        add_header Cross-Origin-Embedder-Policy "require-corp" always

        # Basis-Härtung
        add_header X-Frame-Options "SAMEORIGIN" always;
        add_header X-Content-Type-Options "nosniff" always;
        add_header Referrer-Policy "strict-origin-when-cross-origin" 

        # 5. CACHING STRATEGIE
        # WASM/JS sind unveränderlich (Immutable), HTML muss frisch b
        location ~* \.(wasm|js|css)$ {
            expires 1y;
            add_header Cache-Control "public, immutable";
            # Header müssen im Location-Block wiederholt werden
            add_header Cross-Origin-Opener-Policy "same-origin" alway
            add_header Cross-Origin-Embedder-Policy "require-corp" al
        }

        location ~* \.html$ {
            expires -1;
            add_header Cache-Control "no-store, no-cache, must-revali
            add_header Cross-Origin-Opener-Policy "same-origin" alway
            add_header Cross-Origin-Embedder-Policy "require-corp" al
        }

        # Blockiert Zugriff auf versteckte Dateien (.git, .env)
        location ~ /\. {
            deny all;
        }
    }
}
```

---

5. Docker (Hardened Container)

Für Kubernetes, OpenShift oder moderne CI/CD Pipelines stellen wir ein **Rootless Image** bereit. Dies erfüllt strikte Compliance-Vorgaben in Enterprise-Umgebungen.

Sicherheits-Features

- **Non-Root:** Der Container läuft als User `nginx` (UID 101), nicht als `root`.
- **Minimal Base:** Basiert auf Alpine Linux (< 15 MB Größe).
- **Read-Only Ready:** Kann mit schreibgeschütztem Dateisystem betrieben werden.

Installationsanweisung

1. Legen Sie das `Dockerfile` in das Verzeichnis über Ihrem `wasm`-Ordner.
2. Legen Sie die oben erstellte `nginx.conf` daneben.
3. Build: `docker build -t proedc:3.1 .`
4. Run: `docker run -d -p 80:8080 proedc:3.1`

Konfiguration: `Dockerfile`

```
# STAGE 1: Base Image (Alpine Slim)
FROM nginx:1.25-alpine-slim

# Metadaten für Audit-Scanner
LABEL maintainer="BrainAI Engineering Division"
LABEL description="ProEDC Industrial Suite - High Assurance Container
LABEL version="3.1"

# 1. HARDENING: Root-Rechte entfernen & Verzeichnisse vorbereiten
# Wir passen Rechte an, damit Nginx als Non-Root schreiben kann (PID/
RUN mkdir -p /var/cache/nginx \
             /var/log/nginx \
             /var/run \
    && chown -R nginx:nginx /var/cache/nginx \
                            /var/log/nginx \
                            /var/run \
                            /usr/share/nginx/html \
    && touch /var/run/nginx.pid \
    && chown -R nginx:nginx /var/run/nginx.pid \
    # Standard-Config & Standard-HTML löschen (Angriffsfläche minimie
    && rm -rf /etc/nginx/conf.d/default.conf \
```

```
    && rm -rf /usr/share/nginx/html/*

# 2. CONFIG INJECTION: Optimierte Nginx Config kopieren
COPY nginx.conf /etc/nginx/nginx.conf

# 3. DEPLOYMENT: ProEDC Dateien kopieren
# --chown=nginx:nginx ist wichtig für den Non-Root Betrieb
COPY --chown=nginx:nginx ./wasm /usr/share/nginx/html

# 4. USER SWITCH: Ab hier läuft der Container sicher als "nginx" User
USER nginx

# 5. NETZWERK: Wir nutzen intern Port 8080 (Non-Privileged)
EXPOSE 8080

# 6. HEALTHCHECK: Container überwacht sich selbst
HEALTHCHECK --interval=30s --timeout=3s \
  CMD wget --quiet --tries=1 --spider http://localhost:8080/index.htm

# Startbefehl
CMD ["nginx", "-g", "daemon off;"]
```

🇬🇧 ProEDC Industrial Suite | Server Integration Guide

Version: 1.0

Focus: IIS, Apache, Nginx & Docker Hardening

Target Audience: System Administrators / DevOps / IT-Ops

---

1. General System Requirements

The ProEDC Industrial Suite is a **client-side-only** application. The entire cryptographic logic is executed within the user's browser via WebAssembly

(WASM). No server-side scripting language (such as PHP, Python, ASP.NET) is required.

However, to ensure high performance (LCP < 1s) and secure execution, the web server must fulfill three specific requirements:

A. MIME-Type Configuration

WebAssembly files (`.wasm`) must be served with the correct MIME type: `application/wasm`.

- *Error symptom:* The browser refuses to load the app or displays console errors ("Incorrect response MIME type").

B. Security Headers (COOP / COEP)

For high-performance cryptographic operations, ProEDC utilizes modern memory structures (`SharedArrayBuffer`). Modern browsers (Chrome, Edge, Firefox) require strict cross-origin isolation to enable this feature.

- **Cross-Origin-Opener-Policy:** `same-origin`
- **Cross-Origin-Embedder-Policy:** `require-corp`

C. Compression (Brotli / Gzip)

The `proedc.wasm` binary is optimized, but uncompressed by default. The server should enable **static compression** to drastically reduce the file size for transmission.

---

2. Microsoft IIS (Internet Information Services)

In Windows Server environments, configuration is most easily achieved via a `web.config` file. This instructs IIS to handle the `.wasm` file correctly without altering global server settings.

Installation Instructions

1. Copy the contents of the `wasm` folder into your webroot (e.g., `C:\inetpub\wwwroot\proedc`).
2. Create a file named `web.config` in the same directory.
3. Paste the following XML code into the file.

Configuration: `web.config`

```
<?xml version="1.0" encoding="UTF-8"?>
<configuration>
  <system.webServer>

    <staticContent>
      <remove fileExtension=".wasm" />
      <mimeMap fileExtension=".wasm" mimeType="application/wasm" />
      <remove fileExtension=".json" />
      <mimeMap fileExtension=".json" mimeType="application/json" />
    </staticContent>

    <httpProtocol>
      <customHeaders>
        <add name="Cross-Origin-Opener-Policy" value="same-origin" />
        <add name="Cross-Origin-Embedder-Policy" value="require-corp"

        <add name="X-Content-Type-Options" value="nosniff" />
        <add name="X-Frame-Options" value="SAMEORIGIN" />
        <add name="Strict-Transport-Security" value="max-age=31536000
      </customHeaders>
    </httpProtocol>

    <urlCompression doStaticCompression="true" doDynamicCompression="

    <caching>
      <profiles>
        <add extension=".wasm" policy="CacheUntilChange" kernelCacheP
        <add extension=".js" policy="CacheUntilChange" kernelCachePol
      </profiles>
    </caching>

  </system.webServer>
</configuration>
```

Verification of IIS Installation

To ensure IIS is configured correctly:

1. Open the URL in your browser (Edge/Chrome).
2. Press `F12` to open Developer Tools -> **Network** tab.
3. Reload the page.
4. Click on the `proedc.wasm` file in the network list.
5. Check the **Response Headers**:

- `Content-Type`: Must be `application/wasm`.
- `Cross-Origin-Embedder-Policy`: Must be `require-corp`.

---

3. Apache HTTP Server (.htaccess)

The Apache Web Server remains the standard in many Linux-based intranets and shared hosting environments. Configuration is managed via an `.htaccess` file in the webroot.

Installation Instructions

1. Copy the contents of the `wasm` folder into your webroot (e.g., `/var/www/html/proedc`).
2. Create a file named `.htaccess` in the same directory (do not forget the leading dot!).
3. Paste the provided configuration code.

Important Prerequisites: Ensure that the following modules are enabled in the main configuration (`httpd.conf`) – these are usually enabled by default:

- `mod_headers` (for Security Headers)
- `mod_mime` (for WASM type recognition)
- `mod_deflate` (for compression)
- `mod_expires` (for caching)

How the Configuration Works

- **WASM Isolation:** The `Cross-Origin-Opener-Policy` and `Cross-Origin-Embedder-Policy` headers are set. This allows the browser to utilize `SharedArrayBuffer`, boosting cryptographic performance by up to 30%.
- **Compression:** The `proedc.wasm` file is compressed prior to delivery, reducing transfer times by an average of 60-70%.
- **Lockdown:** Access to internal files (such as `.git` folders or Makefiles) that may have been uploaded accidentally is strictly blocked.

---

4. Nginx (High Performance & Linux)

Nginx is the recommended environment for high-load production deployments. This configuration is aggressively optimized for the rapid delivery of WebAssembly streams.

Installation Instructions

1. Create a file named `nginx.conf` in your configuration directory (or use this snippet in your `sites-available`).
2. Ensure the `root` path correctly points to your `wasm` folder.

Configuration: `nginx.conf` (Optimized)

```
# Worker Processes set to "auto" utilizes all CPU cores for I/O
worker_processes auto;

events {
    worker_connections 1024;
    multi_accept on;
}

http {
    include /etc/nginx/mime.types;
    default_type application/octet-stream;

    # 1. WASM MIME-TYPE SUPPORT (Critical)
    types {
        application/wasm wasm;
```

```
}

# 2. PERFORMANCE TUNING
sendfile on;
tcp_nopush on;
tcp_nodelay on;
keepalive_timeout 65;
server_tokens off; # Hides version info (Security by Obscurity)

# 3. GZIP COMPRESSION
# Reduces .wasm file load time by approx. 60-70%
gzip on;
gzip_vary on;
gzip_proxied any;
gzip_comp_level 6; # Balance between CPU load and compression
gzip_types text/plain text/css application/json application/javas

server {
    listen 80;
    server_name localhost;
    root /usr/share/nginx/html; # Path to ProEDC 'wasm' folder
    index index.html;

    # 4. SECURITY HEADERS (Industrial Hardening)
    # Required for SharedArrayBuffer (WASM Threading Support)
    add_header Cross-Origin-Opener-Policy "same-origin" always;
    add_header Cross-Origin-Embedder-Policy "require-corp" always

    # Baseline Hardening
    add_header X-Frame-Options "SAMEORIGIN" always;
    add_header X-Content-Type-Options "nosniff" always;
    add_header Referrer-Policy "strict-origin-when-cross-origin"

    # 5. CACHING STRATEGY
    # WASM/JS are immutable, HTML must remain fresh.
    location ~* \.(wasm|js|css)$ {
        expires 1y;
        add_header Cache-Control "public, immutable";
        # Headers must be repeated within the location block
        add_header Cross-Origin-Opener-Policy "same-origin" alway
        add_header Cross-Origin-Embedder-Policy "require-corp" al
    }

    location ~* \.html$ {
```

```
            expires -1;
            add_header Cache-Control "no-store, no-cache, must-revali
            add_header Cross-Origin-Opener-Policy "same-origin" alway
            add_header Cross-Origin-Embedder-Policy "require-corp" al
        }

        # Blocks access to hidden files (.git, .env)
        location ~ /\. {
            deny all;
        }
    }
}
```

---

5. Docker (Hardened Container)

For Kubernetes, OpenShift, or modern CI/CD pipelines, we provide a **Rootless Image**. This fulfills strict compliance requirements in enterprise environments.

Security Features

- **Non-Root:** The container runs as the `nginx` user (UID 101), not as `root`.
- **Minimal Base:** Based on Alpine Linux (< 15 MB footprint).
- **Read-Only Ready:** Capable of running with a read-only filesystem.

Installation Instructions

1. Place the `Dockerfile` in the directory above your `wasm` folder.
2. Place the previously created `nginx.conf` alongside it.
3. Build: `docker build -t proedc:3.1 .`
4. Run: `docker run -d -p 80:8080 proedc:3.1`

Configuration: `Dockerfile`

```
# STAGE 1: Base Image (Alpine Slim)
FROM nginx:1.25-alpine-slim
```

```
# Metadata for Audit Scanners
LABEL maintainer="BrainAI Engineering Division"
LABEL description="ProEDC Industrial Suite - High Assurance Container
LABEL version="3.1"

# 1. HARDENING: Remove root privileges & prepare directories
# Adjusting permissions so Nginx can write as a non-root user (PID/Lo
RUN mkdir -p /var/cache/nginx \
             /var/log/nginx \
             /var/run \
    && chown -R nginx:nginx /var/cache/nginx \
                            /var/log/nginx \
                            /var/run \
                            /usr/share/nginx/html \
    && touch /var/run/nginx.pid \
    && chown -R nginx:nginx /var/run/nginx.pid \
    # Remove default config & default HTML (minimize attack surface)
    && rm -rf /etc/nginx/conf.d/default.conf \
    && rm -rf /usr/share/nginx/html/*

# 2. CONFIG INJECTION: Copy optimized Nginx Config
COPY nginx.conf /etc/nginx/nginx.conf

# 3. DEPLOYMENT: Copy ProEDC files
# --chown=nginx:nginx is critical for non-root operation
COPY --chown=nginx:nginx ./wasm /usr/share/nginx/html

# 4. USER SWITCH: From here on, the container runs securely as the "n
USER nginx

# 5. NETWORK: We use internal port 8080 (Non-Privileged)
EXPOSE 8080

# 6. HEALTHCHECK: Container monitors itself
HEALTHCHECK --interval=30s --timeout=3s \
  CMD wget --quiet --tries=1 --spider http://localhost:8080/index.htm

# Start command
CMD ["nginx", "-g", "daemon off;"]
```

© 2026 Sascha Köhne	**✉️ Contact for KYC & Early Access: koehne83@googlemail.com**	**🛡️ ProHash-Verified**
System Architect	BrainAI Inhaber : Sascha Alexander Köhne	Deterministische Integrität